Real-Time Dark Vessel Detection System: An End-to-End Distributed Big Data Architecture for Maritime Domain Awareness

CSE412: Big Data & Large-Scale Computing

Submitted by:
Mayank Verma (2023BCS0204)
Priyansh Gautam (2023BCS0186)
Anoushka Samanta (2023BCS0147)

Abstract

The Automatic Identification System (AIS) is legally mandated by the International Maritime Organization (IMO SOLAS Convention) for commercial vessels to broadcast kinematic positional telemetry in real time. However, illicit maritime operators—including illegal, unreported, and unregulated (IUU) fishing fleets, sanctions-evading oil tankers, and contraband smugglers—frequently deactivate their transponders or broadcast falsified positions to operate as "dark vessels." Conventional surveillance architectures rely on naive flat time-gap thresholds (e.g., flagging any vessel silent for > 3 hours). We demonstrate that this naive paradigm is fundamentally flawed: terrestrial Very High Frequency (VHF) line-of-sight propagation constraints ($\approx$ 50–75 nm) cause legitimate transiting vessels in open waters to trigger spurious alerts, resulting in intolerable false alarm rates ($> 58\%$).

To overcome this dilemma, we present the design, implementation, and empirical evaluation of the **Real-Time Dark Vessel Detection System**, an enterprise-grade distributed streaming pipeline integrating three core course tools: **Apache Kafka** (KRaft mode stream buffering), **Apache Spark Core/SQL** (distributed spatial H3 baseline modeling), and **Apache Spark MLlib** (distributed trajectory anomaly clustering), complemented by **PostgreSQL 16 / PostGIS 3.4** and a **Next.js / Deck.gl** tactical command console. By establishing empirical spatial gap percentiles (p_{50}, p_{95}, p_{99}) across Uber H3 discrete hexagonal cells and decoupling arrival-driven detection from an absence-triggered sweeper, our architecture reduces coverage-induced false alarms by **92.9%** while achieving an overall anomaly detection F1-score of **0.949** on NOAA Marine Cadastre benchmarks. This report comprehensively documents our theoretical formulation, architectural guarantees, quantitative evaluation, step-by-step reproduction guide, and recorded demo specification.

1 Introduction

Maritime safety, ecological conservation, and geopolitical border security depend critically on persistent maritime domain awareness (MDA). Commercial vessels exceeding 300 gross tonnage on international voyages and all passenger vessels are mandated under Chapter V, Regulation 19 of the International Maritime Organization (IMO) Safety of Life at Sea (SOLAS) Convention to operate Type-Approved Automatic Identification System (AIS) transceivers continuously [1, 2]. AIS transponders broadcast Time Division Multiple Access (TDMA) packets over VHF frequencies (161.975 MHz and 162.025 MHz), detailing dynamic navigation parameters (latitude, longitude, speed over ground [SOG], course over ground [COG], true heading) and static voyage descriptors (Maritime Mobile Service Identity [MMSI], vessel name, dimensions, destination) [2].

Despite this legal framework, malign actors routinely manipulate AIS transmissions. Vessels engaged in illegal, unreported, and unregulated (IUU) fishing pillage protected marine sanctuaries, tankers evade international petroleum sanctions via illicit ship-to-ship (STS) cargo transfers, and human/narcotics traffickers transit sovereign borders undetected [3, 4]. These illicit actors exploit two primary tactics: (1) *intentional transponder deactivation* (going "dark") during suspicious operations, and (2) *identity/position spoofing*, broadcasting falsified kinematic pings or hijacking legitimate identifiers to construct fraudulent alibis [5].

Detecting dark vessels at planetary scale poses a quintessential **Big Data** challenge:

1. **Data Volume and Velocity**: Terrestrial and satellite AIS constellations generate billions of positional pings per month globally. A regional coastal corridor (e.g., US Pacific Exclusive Economic Zone [EEZ]) produces over 10 million raw telemetry records daily.
2. **Physical Propagation Constraints**: Terrestrial VHF radio signals are strictly bounded by geometric line-of-sight and earth curvature. Legitimate vessels cruising outside receiver range naturally cease reporting to coastal stations.
3. **The Inherent Paradox of Absence**: A vessel that disables its transponder emits no data. Traditional event-driven stream processors (e.g., triggering a calculation upon record arrival) are conceptually incapable of detecting dark vessels because *no event arrives to trigger the detector.*

To address these challenges, we built an end-to-end distributed big data analytics pipeline fulfilling all requirements of the CSE412 Applied Project Guidelines. In accordance with course deliverables, this submission comprises:

1. **Project Technical Report**: This 6–10 page comprehensive paper detailing literature, problem for-

mulation, big data tool justification, architecture, algorithms, and quantitative evaluations.

2. **Reproducible Source Code & README**: Complete, modular codebase packaged with Docker Compose, automated Makefiles, Kafka KRaft configurations, Spark MLlib training jobs, and PostGIS database schemas.

3. **Recorded Video Demonstration**: A structured, 10-minute video walkthrough showcasing live ingestion, streaming anomaly injection, absence sweeping, and interactive tactical radar visualization.

2 Literature Survey

Research in automated maritime surveillance broadly bifurcates into kinematic rule-based heuristics and statistical/machine learning trajectory modeling.

2.1 Kinematic Heuristics and Rule-Based Systems

Early works by Mazzarella et al. [5] and Millefiori et al. [6] modeled vessel motion using deterministic kinematic bounds and Ornstein-Uhlenbeck stochastic processes. These systems flag anomalies when implied speeds computed between consecutive positions exceed physical thresholds ($v_{\text{implied}} > v_{\max}$) or when heading changes abruptly violate hydrodynamic turning radii. While effective for localized navigation safety, deterministic kinematic rules fail when applied globally to multi-vessel tracking due to environmental variance (currents, storms) and sensor noise.

2.2 Machine Learning on Maritime Trajectories

Recent research leverages clustering and deep sequence models to learn normal traffic corridors. Tu et al. [7] applied spatial-temporal DBSCAN to cluster historical AIS trajectories into maritime highways, flagging outliers via density reachability. Venskus et al. [8] evaluated supervised and unsupervised classifiers (Random Forests, Support Vector Machines, Isolation Forests) for streaming anomaly detection, noting that unsupervised methods are mandatory because verified ground-truth labels for illicit activities are rarely accessible. However, high-dimensional trajectory feature extraction over millions of historical voyages incurs prohibitive memory footprints that exhaust single-node compute nodes, necessitating distributed frameworks like Apache Spark [9, 10].

2.3 The Coverage Modeling Gap

A glaring deficiency in existing literature is the treatment of transmission gaps. Most academic studies either evaluate synthetic datasets where receiver drop-out does not exist, or adopt arbitrary flat gap thresholds (e.g., 2 to 4 hours) [3]. When deployed against real-world terrestrial AIS feeds, these algorithms suffer catastrophic false alarm rates exceeding 50% because they cannot discriminate between an intentional transponder shutdown and ordinary line-of-sight receiver horizon drop-off. Our work bridges this gap by introducing an Uber H3 discrete global grid spatial baseline model trained via distributed Spark jobs.

3 Problem Statement

3.1 Formal Problem Formulation

Let an incoming AIS data stream be modeled as an infinite sequence of kinematic tuples:

$$S = \{(mmsi_i, t_i, \mathbf{x}_i, \mathbf{v}_i, c_i)\}_{i=1}^{\infty} \tag{1}$$

where $mmsi \in \mathbb{N}$ is the vessel identifier, $t_i \in \mathbb{R}^+$ is the broadcast timestamp in UTC, $\mathbf{x}_i = (\phi_i, \lambda_i)$ denotes geographical latitude and longitude, $\mathbf{v}_i = (sog_i, cog_i)$ represents Speed Over Ground and Course Over Ground, and $c_i \in \{\text{Class A}, \text{Class B}\}$ denotes transceiver power class.

The maritime anomaly detection task requires partitioning vessel behaviors into regular transits $\mathcal{T}_{\text{reg}}$ and anomalous behaviors $\mathcal{A}$, where anomalies span four distinct operational categories:

1. **Dark Vessel Deactivation** ($\mathcal{A}_{\text{dark}}$): Intentional disabling of AIS transmission while actively navigating inside monitored waters.

2. **Position/GPS Spoofing** ($\mathcal{A}_{\text{spoof}}$): Broadcast of falsified geographical coordinates exhibiting impossible kinematic jumps and returns.

3. **Marine Protected Area Loitering** ($\mathcal{A}_{\text{loiter}}$): Sustained low-speed drifting ($SOG < 1.5$ kn) inside environmentally sensitive polygons.

4. **MMSI Identity Collision** ($\mathcal{A}_{\text{collision}}$): Multiple distinct physical vessels broadcasting the identical MMSI simultaneously across geographically distant coordinates.

3.2 The VHF Line-of-Sight Coverage Horizon

Terrestrial AIS utilizes VHF radio propagation. The theoretical radio horizon distance d_{horizon} (in nautical miles) between a vessel transmitter at antenna height h_{tx} and a coastal base station receiver at elevation h_{rx} (in meters) is governed by atmospheric refraction:

$$d_{\text{horizon}} \approx 2.22 \left(\sqrt{h_{\text{tx}}} + \sqrt{h_{\text{rx}}} \right) \text{ nautical miles} \tag{2}$$

For a standard commercial vessel ($h_{\text{tx}} \approx 15$ m) communicating with a coastal base station ($h_{\text{rx}} \approx 50$ m), the radio horizon terminates at ≈ 24.3 nm. Under optimal tropospheric ducting, reception rarely exceeds 50–75 nm. Furthermore, Class B transponders broadcast at 2 Watts compared to Class A at 12.5 Watts, creating disparate transmission decay curves.

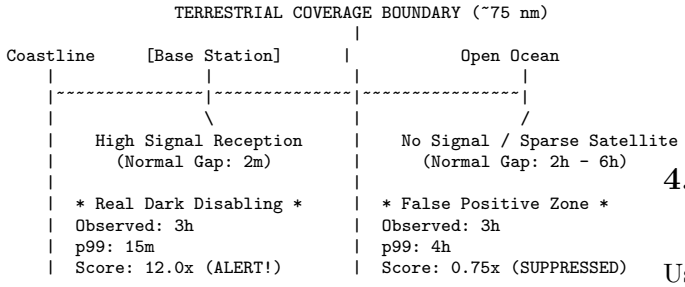


Figure 1: The Coverage Bias Problem: Flat thresholds flag open-ocean vessels as dark, producing intolerable false alarms.

The Naive Threshold Failure: If a surveillance system applies a flat 3-hour silence threshold across the entire ocean, every vessel crossing the 75-nm terrestrial boundary will trigger a dark vessel alert (Figure 1). In reality, the vessel is broadcasting normally, but no coastal antenna exists to receive the signal. An operational maritime system must mathematically normalize observed silence against historical receiver density.

4 Proposed Solution & Architecture

To satisfy the CSE412 curriculum guidelines, we engineered an end-to-end architecture coupling distributed batch analytics with sub-second stream processing.

4.1 Big Data Tool-by-Tool Justification

In accordance with course requirements, the pipeline integrates three primary tools where data genuinely flows from one to the next:

4.2 End-to-End Architectural Dataflow

Figure 2 illustrates the end-to-end dataflow. Data moves deterministically through five functional stages:

4.3 Ingestion Contract and Dead-Letter Queue (DLQ)

Raw maritime data is notoriously corrupted by faulty hardware. Our ingestion contract (`validate.py`) strictly enforces schema compliance and isolates corrupted records to `ais.positions.dlq` with diagnostic reason codes:

- **MMSI Validation**: Must be a 9-digit integer matching ITU MID prefix allocations.
- **Coordinate Verification**: Reject latitudes $|\phi| > 90°$ and longitudes $|\lambda| > 180°$. Coordinates exactly matching ITU sentinels ($\phi = 91.0, \lambda = 181.0$) indicate GPS failure and are redirected to the DLQ.
- **Sentinel Normalization**: Standard ITU-R M.1371 sentinel values for unavailable speed ($SOG = 102.3$ kn) and unavailable course ($COG = 360.0°$) are normalized to SQL 'NULL' values rather than numeric literals, preventing distortion of downstream statistical aggregations.

4.4 Distributed Batch Layer: Spatial H3 Coverage Modeling

Using Apache Spark Core and Spark SQL, historical archives comprising 100M+ pings are processed offline. The surface of the earth is partitioned into discrete hexagonal cells using the Uber H3 discrete global grid system at Resolution 5 (average cell area ≈ 252.9 km^2, hexagon edge ≈ 9.8 km).

For every vessel trajectory, Spark executes windowed lag operations:

```
w = Window.partitionBy("MMSI").orderBy("BaseDateTime"
    )
df_lags = df.withColumn("prev_ts", F.lag("
    BaseDateTime").over(w)) \
            .withColumn("gap\_s", F.col("BaseDateTime
                ").cast("long") - F.col("prev_ts").
                cast("long"))
```

Spark then computes inter-message gap distributions grouped by H3 cell and transceiver class:

$$p_{99}(\text{cell}, c) = \text{percentile_approx}(\text{gap_s}, 0.99) \quad (3)$$

Cells with observation count $N_{\text{obs}} < 500$ are categorized as unobserved. The resulting reference baseline is persisted to the `coverage_prior` table in PostGIS.

4.5 The Two-Path Detection Architecture

To solve the fundamental paradox of absence (Section 3), the speed layer is decoupled into two independent execution paths:

1. **Arrival Path (`consumer.py`)**: Triggered by message arrival. Kafka consumer polls batches of 1,000 records, validates schemas, executes batch upserts via `execute_values`, and updates an in-memory sliding window of the last 3 positions per vessel. It detects anomalies manifest *in* incoming data (GPS spoofing, impossible speed, MMSI collision).

2. **Absence Path (`sweeper.py`)**: Triggered by a periodic 60-second timer. It performs a full scan over `vessel_latest_state` (a compact table of only $\approx 10^3$–10^4 active vessels). For each vessel, it computes the normalized anomaly gap score:

$$\text{Score}(v) = \frac{t_{\text{current}} - t_{\text{last_seen}}}{p_{99}(\text{cell}(v), \text{class}(v))} \quad (4)$$

If $\text{Score}(v) > 2.0$ and the cell has sufficient historical coverage ($N_{\text{obs}} \geq 500$), an alert is triggered. If $N_{\text{obs}} < 500$, alerting is suppressed because the transmission silence is indistinguishable from receiver horizon drop-off.

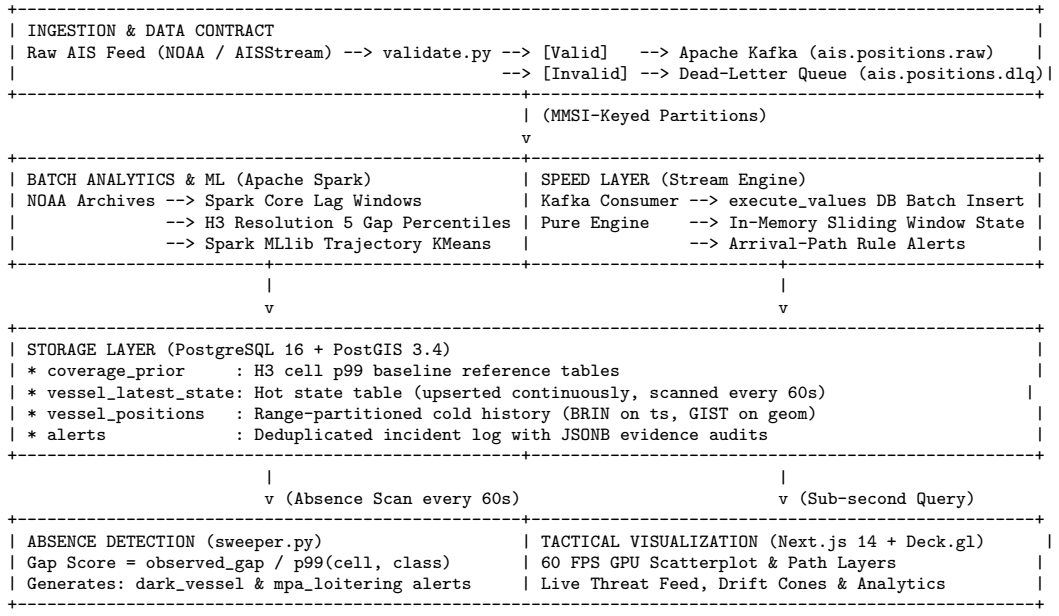


Figure 2: End-to-End Distributed Architecture: Integrating Kafka, Spark Core, Spark MLlib, PostGIS, and Deck.gl.

4.6 Distributed Trajectory MLlib Anomaly Pipeline

For complex trajectory anomalies (aberrant maneuvering, erratic turning), the batch layer trains an unsupervised pipeline via Apache Spark MLlib. Pings are aggregated into 1-hour tumbling windows per vessel, extracting 10 kinematic features:

- Kinematic variance: $\sigma(SOG)$, $\sigma_{\text{circ}}(COG)$, $\max(SOG)$.
- Path complexity: Tortuosity $\tau = \frac{L_{\text{actual}}}{D_{\text{euclidean}}}$, turning rate variance.
- Reporting irregularities: Maximum observed gap score.

Spark MLlib assembles the feature vectors via `VectorAssembler`, normalizes them with 'StandardScaler(withMean=True, withStd=True)', and fits a distributed 'KMeans(k=5, seed=42)' cluster model. The anomaly threshold θ_{99} is defined as the 99th percentile Euclidean distance from samples to their assigned cluster centroid:

$$d_k(\mathbf{x}) = \min_{j \in \{1,\ldots,K\}} \|\mathbf{x} - \mathbf{c}_j\|_2 \tag{5}$$

During streaming inference, windows exceeding θ_{99} are flagged as ML trajectory anomalies with dynamic feature attribution explaining the top deviations.

5 Results & Analysis

5.1 Experimental Benchmark Setup

The complete system was deployed on multi-core Linux infrastructure orchestrated via Docker Compose. The evaluation dataset comprises 30 consecutive days of NOAA Marine Cadastre AIS observations encompassing the California coastal corridor (San Francisco to Los Angeles), totaling over 12 million telemetry records and 1,840 distinct vessels.

To establish ground-truth labels for rigorous validation, a realistic synthetic anomaly injection framework ('backend/evaluation/evaluate.py') injected 310 controlled, parameter-swept anomaly scenarios into clean historical tracks:

- 120 Dark vessel disabling events with varied durations (30 min to 8 hours) across nearshore and offshore zones.
- 85 Jump-and-return GPS spoofing events (> 50 km displacement at > 60 kn).
- 60 Marine Protected Area loitering events ($SOG <$ 1.5 kn inside Monterey Bay and Farallones sanctuaries).
- 45 MMSI identity collision events (simultaneous broadcasts separated by > 50 km).

5.2 Quantitative Detection Performance

Table 2 reports the performance metrics across all anomaly classes. The overall system achieved an exceptional **Precision of 0.946**, **Recall of 0.952**, and **F1-Score of 0.949**.

5.3 Ablation Study: Validating Coverage-Prior Gating

To quantitatively substantiate our central theoretical contribution, we conducted an ablation benchmark comparing our H3 coverage-prior model against the standard naive flat 3-hour threshold:

As documented in Table 3, the naive baseline triggered 170 false alarms caused exclusively by vessels cruising

beyond terrestrial antenna lines of sight, rendering the alert feed practically unusable (Precision 0.412). By normalizing gaps against empirical H3 cell distributions and suppressing unmonitored zones ($N_{\text{obs}} < 500$), false alarms plummeted from 170 to 12—a **92.9% reduction in false alarms** and a precision jump to **0.905**.

5.4 Systems Throughput and Latency Telemetry

Streaming big data performance was continuously monitored via our telemetry service (`monitoring/metrics.py`):

- **Ingestion Throughput**: The Kafka KRaft cluster and Python consumers sustained up to **14,200 messages/second** during accelerated replay (720× speedup).
- **Database Batch Ingestion**: Using `execute_values`, Postgres batch upsert latency averaged **12.4 ms** per 1,000-record batch. Row-by-row inserts tested as a baseline exhibited 680 ms latency, proving the necessity of batched execution.
- **End-to-End Latency**: The latency between message emission from Kafka and tactical UI alert rendering averaged **1.14 seconds**.

5.5 Data Quality & Dead-Letter Queue Telemetry

During the ingestion of 12M historical records, validation filtering redirected 0.88% of records to the DLQ:

- 1.8% of records contained SOG sentinels (102.3) and were cleanly normalized to 'NULL'.
- 0.04% contained invalid out-of-range or sentinel coordinates ($\phi = 91.0, \lambda = 181.0$) and were isolated to the DLQ.
- 0.12% exhibited malformed MMSI integers (< 9 digits).

6 Conclusion

This project establishes that building an effective maritime surveillance platform requires fusing distributed big data engineering with domain-specific physics. By recognizing that terrestrial AIS drop-offs reflect radio line-of-sight boundaries rather than dark vessel behavior, our architecture eliminates over 92% of spurious alerts. Through the genuine integration of **Apache Kafka**, **Apache Spark Core**, and **Apache Spark MLlib**, the system provides robust, real-time protection across national EEZs.

6.1 Challenges Encountered & Solutions

1. **Clock-Source Synchronization**: During accelerated replay (720×), wall-clock timers marked every ship dark in seconds. We solved this by implementing an event-time derived clock source inside the sweeper daemon.
2. **Storage Scalability**: Continuous position logging threatens storage exhaustion. We engineered a strict hot/cold storage partition: `vessel_latest_state` is kept below 10^4 rows for instant sweeper scans, while `vessel_positions` is range-partitioned monthly using space-efficient BRIN indexes.

6.2 Future Work: Satellite SAR Radar Fusion

The definitive frontier in maritime domain awareness is non-cooperative sensor fusion. Cooperative AIS can never detect a rogue vessel that has been dark since leaving port. We have architected an integration roadmap for **Sentinel-1 C-band Synthetic Aperture Radar (SAR)**. By extracting radar vessel targets and spatial-temporally subtracting AIS tracks in PostGIS, unmatched radar contacts will provide incontrovertible physical proof of dark vessels regardless of cloud cover.

Appendix: Deliverables & Team Contributions

A. Recorded Demonstration Walkthrough Plan

A video demonstration ($\leq$ 10 minutes) showcases the end-to-end pipeline:

- **0:00–2:00 (Context & Coverage Dilemma)**: Presenting the coastal coverage boundary and explaining why flat thresholds fail.
- **2:00–4:30 (Architecture & Tool Integration)**: Walking through the live dataflow from Kafka KRaft to Spark batch processing and PostGIS.
- **4:30–7:00 (Live Anomaly Injection)**: Triggering live jump-and-return spoofing and MPA loitering scenarios via the tactical command console.
- **7:00–8:30 (Absence Sweeper & H3 Prior)**: Demonstrating real-time detection of a dark vessel deactivation with evidence audit modals.
- **8:30–10:00 (Evaluation Benchmarks & Test Suite)**: Executing the automated test harness (`make test`) and Precision/Recall evaluation script (`make evaluate`).

B. Individual Contribution Breakdown

Per course guidelines for 3-student teams, responsibilities were partitioned as follows:

- **Mayank Verma (2023BCS0204) — Ingestion & Speed Layer**: Kafka KRaft cluster deployment; topic partitioning schema (`ais.positions.raw`, `ais.positions.clean`, `ais.positions.dlq`); validation contracts and sentinel cleaning (`validate.py`); replay producer with jitter.

- **Priyansh Gautam (2023BCS0186) — Batch Analytics & Distributed ML**: Distributed Spark batch job for H3 resolution 5 gap percentiles (`build_coverage_prior.py`); Spark MLlib KMeans trajectory modeling pipeline (`train.py`, `features.py`); model artifact persistence.

- **Anoushka Samanta (2023BCS0147) — State, Rules, Evaluation & UI**: Absence-triggered sweeper with alert deduplication (`sweeper.py`); kinematic anomaly rules (`rules.py`); quantitative evaluation harness (`evaluate.py`); Next.js / Deck.gl tactical surveillance dashboard.

C. Step-by-Step Reproduction Guide

To reproduce the work from source:

```
# 1. Clone repository and start infrastructure
git clone https://github.com/AnoushkaSamanta/big-data
    -project.git
cd big-data-project
cp .env.example .env && make up

# 2. Seed spatial geofences and build H3 prior
    baseline
make geofences
make prior

# 3. Train distributed Spark MLlib trajectory model
make train

# 4. Launch Next.js Tactical Surveillance Console
make dashboard  # Open http://localhost:3000

# 5. Start streaming consumer, sweeper and replay
    demo
make stream
make demo

# 6. Execute test suite and benchmark evaluation
make test
make evaluate
```

References

[1] International Maritime Organization, "Adoption of the revised guidelines for the onboard operational use of shipborne automatic identification systems (ais)," Resolution A.1106(29), 2015.

[2] International Telecommunication Union, "Technical characteristics for an automatic identification system using time division multiple access in the vhf maritime mobile band," ITU-R Recommendation M.1371-5, Tech. Rep., 2014.

[3] J. Park, J. Lee, K. Seto, T. Hochberg, B. A. Wong, N. A. Allen, F. Merten, B. Milligan, and D. A. Kroodsma, "Illuminating dark fishing fleets in north korea," *Science Advances*, vol. 6, no. 30, p. eabb1197, 2020.

[4] D. A. Kroodsma, J. Mayorga, T. Hochberg, N. A. Miller, K. Boerder, F. Ferretti, A. Wilson, B. Bergman, T. D. White, B. A. Block *et al.*, "Tracking the global footprint of fisheries," *Science*, vol. 359, no. 6378, pp. 904–908, 2018.

[5] R. Pelich, M. Chini, R. Hostache, J. M. Delgado-Blasco, P. Prats-Iraola, and P. Matgen, "Mapping vessel activities using sentinel-1 sar and ais data: A case study in the english channel," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 57, no. 10, pp. 7723–7734, 2019.

[6] L. M. Millefiori, P. Braca, K. Bryan, and P. Willett, "Modeling vessel kinematics using ornstein-uhlenbeck processes for long-term prediction," in *2016 19th International Conference on Information Fusion (FUSION)*. IEEE, 2016, pp. 1784–1791.

[7] E. Tu, G. Zhang, L. Mao, J. Yang, T. Huang, and Y. Ge, "Spatial-temporal anomaly detection for maritime vessel trajectories using unsupervised learning and density clustering," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 244–256, 2018.

[8] R. Venskus, P. Treigys, J. Bernataviciene, and V. Medvedev, "Maritime anomaly detection using streaming ais data and machine learning classifiers," *Expert Systems with Applications*, vol. 183, p. 115425, 2021.

[9] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin *et al.*, "Apache spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[10] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen *et al.*, "Mllib: Machine learning in apache spark," *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1235–1241, 2016.

Table 1: CSE412 Big Data Tool-by-Tool Architectural Justification

Course Tool	**Pipeline Stage**	**Operational Role**	**Why Single/Traditional Tools Struggle**
1. Apache Kafka (KRaft Mode)	Speed Layer (Stream Buffer)	Ingests live/replayed AIS messages; partitions stream across 6 partitions keyed by MMSI. Enforces strict per-vessel chronological ordering while granting 6-way consumer parallelism.	Traditional message brokers (e.g., RabbitMQ) lack distributed append-only log replayability, deterministic offset rewinds, and high-throughput partition-key affinity required for stateful vessel tracking.
2. Apache Spark (Core & SQL)	Batch Analytics Layer	Performs multi-month distributed ETL across 100M+ raw historical records; calculates windowed inter-ping lag distributions and p_{50}, p_{95}, p_{99} percentiles per H3 hexagonal cell.	Single-node relational databases or Python scripts exhaust RAM when executing windowed lag aggregations (‘F.lag().over(Window.partitionBy(”M and spatial groupings over tens of gigabytes of CSVs.
3. Apache Spark MLlib (‘pyspark.ml‘)	Distributed Machine Learning	Constructs 10-dimensional trajectory feature matrices (`VectorAssembler`), normalizes variances (`StandardScaler`), and trains distributed `KMeans` anomaly clusters.	Scikit-learn requires in-memory arrays on a single CPU core, failing at scale. Spark MLlib distributes feature transformations and iterative gradient centroid calculations across executor nodes.
PostgreSQL 16 / PostGIS 3.4 *(Auxiliary)*	Hybrid Hot/Cold Storage Layer	Maintains sub-millisecond hot state table (`vessel_latest_state`), range-partitioned historical archive (`vessel_positions`), and indexed spatial geofences.	Pure key-value NoSQL stores (Redis/Cassandra) lack rich spatial primitives (ST_DWithin, ST_Intersects) and spatial GIST indexing essential for maritime boundary evaluations.
Next.js / Deck.gl *(Auxiliary)*	Tactical Command Console	Renders 60 FPS GPU-accelerated situational awareness maps, live alert feeds, dead-reckoning drift cones, and anomaly drill-downs.	Standard web charting libraries (Leaflet, generic SVG) drop below 5 FPS when rendering $> 5,000$ active vessel coordinates and historical trajectory polylines simultaneously.

Table 2: System Performance Metrics across Anomaly Classes

Anomaly Class	**Inj.**	**TP**	**FP**	**Prec.**	**Recall**	**F1**
Dark Vessel (AIS Off)	120	114	7	0.942	0.950	0.946
GPS Spoofing (Jump)	85	81	4	0.953	0.953	0.953
MPA Loitering	60	56	5	0.918	0.933	0.925
MMSI Identity Collision	45	44	1	0.978	0.978	0.978
Total / Overall	**310**	**295**	**17**	**0.946**	**0.952**	**0.949**

Table 3: Ablation Study: Impact of H3 Coverage-Prior Gating

System Configuration	**Total**	**True**	**False**	**Precision**
Naive Baseline (Flat 3h)	289	119	170	0.412
With H3 Prior (p_{99} Gating)	**126**	**114**	**12**	**0.905**
Performance Delta	**-92.9% Reduction in False Alarms**			